

Lab Report: Week 3 - Magnetic field in a Solenoid

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-08-12

Imports and Functions

All calculations were done in Python. Starting with imports:

```
from dataclasses import dataclass
import matplotlib.pyplot as plt
from uncertainties import ufloat
import numpy as np
from scipy import odr
import pandas as pd
```

Defining the physical constants and parameters for the solenoid setup:

```
N = 700 # no. of turns in the solenoid
mu_0 = 4 * np.pi * 10**-7 # permeability of free space (Wb / A m)

L_solenoid = ufloat(0.15, 0.002) # length of solenoid in (m)
R_solenoid = ufloat(0.015, 0.001) # radius of solenoid (m)
I_solenoid = ufloat(2.0, 0.02) # solenoid current (A)

V_0 = ufloat(2.50, 0.01) # measured output voltage with no magnetic field (V)
S_sensor = ufloat(3.13, 0.09) * 0.001 / 0.0001 # sensitivity in V/T (converted
from 3.13 mV/G)
```

Since the magnetic field of a finite solenoid follows a non-linear relationship with respect to axial distance, standard linear regression is insufficient. our measurements have uncertainties in both distance and measured voltage/magnetic field, so I made a custom non-linear `odr_fit` function using `scipy.odr`. It extracts nominal values and errors, fits a custom theoretical model, and returns the optimized parameters along with fit metrics like R^2 , RMSE, and the reduced chi-squared statistic (χ^2_ν).

```

@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    rmse: float
    reduced_chi_squared: float
    converged: bool
    x_nom: np.ndarray
    y_nom: np.ndarray
    x_err: np.ndarray
    y_err: np.ndarray

def solenoid_model(p, d):
    scale, offset = p
    L = L_solenoid.n
    R = R_solenoid.n
    I = I_solenoid.n

    term1 = (d + L / 2) / np.sqrt((d + L / 2) ** 2 + R ** 2)
    term2 = (d - L / 2) / np.sqrt((d - L / 2) ** 2 + R ** 2)
    B_z = (mu_0 / (4 * np.pi)) * (2 * np.pi * N * I / L) * (term1 - term2)

    return scale * B_z + offset

def odr_fit(x: list[ufloat], y: list[ufloat]) -> odr_fit_result:
    x_nom = np.array([v.n for v in x])
    x_err = np.array([v.s for v in x])
    y_nom = np.array([v.n for v in y])
    y_err = np.array([v.s for v in y])

    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    model = odr.Model(solenoid_model)
    beta0 = [1.0, 0.0]

    odr_result = odr.ODR(data, model, beta0=beta0).run()

    parameters = odr_result.beta
    parameter_errors = odr_result.sd_beta
    converged = odr_result.info in (1, 2, 3)
    if not converged:
        print(f"WARNING: ODR did not clearly converge (info={odr_result.info})")

```

```

y_pred = solenoid_model(parameters, x_nom)
residuals = y_nom - y_pred
ss_res = np.sum(residuals**2)
ss_tot = np.sum((y_nom - np.mean(y_nom)) ** 2)
r_squared = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan
rmse = np.sqrt(np.mean(residuals**2))
reduced_chi_squared = odr_result.res_var

return odr_fit_result(
    parameters=parameters,
    parameter_errors=parameter_errors,
    r_squared=r_squared,
    rmse=rmse,
    reduced_chi_squared=reduced_chi_squared,
    converged=converged,
    x_nom=x_nom,
    y_nom=y_nom,
    x_err=x_err,
    y_err=y_err,
)

```

To visualize our results, a custom plot function produces both the experimental data with the fitted theoretical curve and a residual plot.

```

def plot(x, y, sx, sy, fit_result, xlabel, ylabel, title, filename):
    x_fit = np.linspace(min(x), max(x), 200)
    y_fit = solenoid_model(fit_result.parameters, x_fit)
    residuals = y - solenoid_model(fit_result.parameters, x)

    fig, (ax1, ax2) = plt.subplots(
        2, 1, figsize=(7, 7), sharex=True,
        gridspec_kw={"height_ratios": [3, 1]}
    )

    ax1.errorbar(x, y, xerr=sx, yerr=sy, fmt="o", capsize=3,
        label="Experimental Data (Hall Sensor)")
    ax1.plot(x_fit, y_fit, "k--",
        label=f"Fit ( $\chi^2_v = \{fit\_result.reduced\_chi\_squared:.2f\}$ ")
    ax1.set_ylabel(ylabel)
    ax1.set_title(title)
    ax1.grid(True, alpha=0.6)
    ax1.legend()

```

```

ax2.errorbar(x, residuals, yerr=sy, fmt="o", capsize=3, color="crimson")
ax2.axhline(0, color="black", linestyle="--")
ax2.set_xlabel(xlabel)
ax2.set_ylabel("Residual (T)")
ax2.grid(True, alpha=0.6)

plt.tight_layout()
plt.savefig(filename, dpi=300)
plt.show()

```

This function checks the reduced chi-squared (χ^2_ν) value to determine if the experimental data agrees with the theoretical model:

```

def chi_squared_test(red_chi2):
    print(f"reduced chi-squared ( $\chi^2_\nu$ ): {red_chi2:.3f}")
    if 0.5 <= red_chi2 <= 1.5:
        print("results AGREE")
    elif red_chi2 < 0.5:
        print("chi_v^2 << 1: uncertainties may be overestimated")
    else:
        print("chi_v^2 >> 1: results DISAGREE")
    return red_chi2

```

Pre-lab Questions

Question 1-

Using equation 2 from the notes:

$$|B| = \frac{|V - V_0|}{S}$$

Given:

$$V = 2.18 \text{ V}$$

$$V_0 = 2.55 \text{ V}$$

$$S = 3.13 \text{ mV/G} = \frac{3.13 \times 10^{-3} \text{ V}}{(10^{-4} \text{ T})} = 31.3 \text{ V/T}$$

Substituting:

$$|B| = \frac{|2.18 \text{ V} - 2.55 \text{ V}|}{31.3 \text{ V/T}} = \frac{0.37 \text{ V}}{31.3 \text{ V/T}} \approx 0.01182 \text{ T} = 11.82 \text{ mT}$$

Question 2-

The equation for the reduced chi-squared value (χ^2_ν) is given by:

$$\chi^2_\nu = \frac{1}{\nu} \sum_{i=1}^N \left(\frac{y_i - y_{\text{model}}(x_i)}{\sigma_{y,i}} \right)^2$$

where:

χ^2_ν : The reduced chi-squared statistic.

ν : The number of degrees of freedom, typically defined as $\nu = N - m$, where N is the total number of data points and m is the number of fitted parameters.

N : Total number of experimental observations/data points.

y_i : The measured value of the dependent variable for the i -th data point.

$y_{\text{model}}(x_i)$: The theoretical or expected value predicted by the model at the i -th data point.

$\sigma_{y,i}$: The experimental uncertainty (standard error) associated with the i -th measurement y_i .

Aim:

To measure the magnetic field strength along the axis of a solenoid using a Hall effect sensor and compare the spatial distribution against theoretical predictions.

Theory:

Theoretical Model

The theoretical magnetic field B_z along the central axis of a finite solenoid of length L , radius R , with N turns carrying current I is given by:

$$B_z = \frac{\mu_0 N I}{2L} \left(\frac{d + L/2}{\sqrt{(d + L/2)^2 + R^2}} - \frac{d - L/2}{\sqrt{(d - L/2)^2 + R^2}} \right)$$

where d represents the axial distance from the center of the solenoid.

Sensor Conversion

The Hall effect sensor translates magnetic fields into a transverse voltage. The magnitude of the magnetic field is determined from the measured voltage V_{meas} and the zero-field baseline voltage V_0 using the sensor sensitivity S :

$$|B| = \frac{|V_{\text{meas}} - V_0|}{S}$$

Procedure

Results